federated-learning security cryptography signguard ecdsa byzantine-robustness

SignGuard: Designing Cryptographic Defenses for Federated Learning

Deep dive into SignGuard's architecture — combining ECDSA digital signatures, multi-factor anomaly detection, and time-decay reputation scoring to defend federated learning against poisoning attacks.

January 28, 2026 15 min read Azka

SignGuard is my attempt to solve a fundamental problem in federated learning: how do we trust that client updates are genuine and tamper-free?

This post walks through SignGuard’s design decisions, implementation challenges, and why I believe cryptographic defenses are the future of FL security.

The Problem

In standard federated learning:

Clients send model updates (gradients)

Server aggregates them (FedAvg, Krum, etc.)

Updated model is redistributed

The vulnerability: Step 2 assumes all updates are well-intentioned. A Byzantine client can send arbitrary updates:

```
# Malicious client
malicious_update = genuine_update * 1000  # Scaling attack
# or
malicious_update = -genuine_update         # Sign flipping
```

Existing defenses (Krum, clustering) try to detect these statistically. But they're fragile—adaptive attacks bypass them.

SignGuard's Thesis

Instead of detecting bad updates, prevent them.

Use cryptography and statistical analysis to ensure:

- Updates come from authenticated clients
- Updates haven't been tampered with in transit
- Clients can't deny sending an update (non-repudiation)
- Updates satisfy quality constraints through multi-factor anomaly detection

Design Architecture

SignGuard has three layers:

```
┌──────────────────────────────────────────────────┐
│      Layer 3: Multi-Factor Anomaly Detection     │
│    (Magnitude, direction, and loss-based analysis)  │
├──────────────────────────────────────────────────┤
│      Layer 2: Time-Decay Reputation Scoring      │
│         (Adaptive trust based on behavior)       │
├──────────────────────────────────────────────────┤
│    Layer 1: Digital Signatures (ECDSA P-256)     │
│    (Authenticate and integrity-protect updates)  │
└──────────────────────────────────────────────────┘
```

Layer 1: ECDSA Signature Verification

Initial Handshake

When a client joins the FL system:

```python
class ClientRegistration:
    def register_client(self, client_id: str) -> KeyPair:
        # Generate ECDSA keypair
        private_key = ec.generate_private_key(ec.SECP256K1())
        public_key = private_key.public_key()
```

```
        # Store public key server-side
        self.public_keys[client_id] = public_key

        # Return private key to client (secure channel!)
        return KeyPair(private_key, public_key)
```

Signing Process

Before sending an update:

```
def sign_update(update: np.ndarray, private_key) -> bytes:
    # Serialize update deterministically
    update_bytes = update.tobytes()

    # Create signature
    signature = private_key.sign(
        update_bytes,
        ec.ECDSA(hashes.SHA256())
    )

    return signature
```

Verification Process

On the server side:

```
def verify_update(
    client_id: str,
    update: np.ndarray,
    signature: bytes
) -> bool:
    # Get client's public key
    public_key = self.public_keys.get(client_id)
    if not public_key:
        return False  # Unknown client

    # Verify signature
    try:
        public_key.verify(
            signature,
```

```
            update.tobytes(),
            ec.ECDSA(hashes.SHA256())
        )
        return True
    except InvalidSignature:
        return False
```

Signature Performance

ECDSA on secp256k1:

- **Signing**: ~0.5ms per update
- **Verification**: ~1ms per update
- **Signature size**: 64 bytes

For 100 clients per round: ~100ms overhead—acceptable for most FL scenarios.

Security Considerations

- **Private key never leaves client**: Registration happens once, key is stored locally
- **Secure channel**: Registration itself uses TLS
- **Key rotation**: Keys expire and rotate periodically

Layer 2: Multi-Factor Anomaly Detection

Signature verification alone isn't sufficient — authenticated clients can still send malicious updates. This layer applies three complementary anomaly detection techniques:

Magnitude Analysis

The L2 norm of each update is compared against a dynamic baseline:

```
def magnitude_analysis(update: np.ndarray, threshold: float) -> bool:
    update_norm = np.linalg.norm(update)
    return update_norm < threshold
```

Updates significantly larger than expected are flagged. The threshold adapts based on historical data from each client, accounting for natural variation.

Direction Analysis

The cosine similarity between each update and the global model direction:

```
def direction_analysis(update: np.ndarray, global_gradient: np.ndarray,
threshold: float) -> bool:
    similarity = np.dot(update, global_gradient) / (np.linalg.norm(update) *
np.linalg.norm(global_gradient))
    return similarity > threshold
```

Malicious updates often push in divergent directions. This catches backdoor attempts and targeted model poisoning.

Loss-based Analysis

Clients report their training loss alongside their update. The server validates this:

```
def loss_analysis(update: np.ndarray, reported_loss: float, expected_loss:
float, epsilon: float) -> bool:
    return abs(reported_loss - expected_loss) < epsilon
```

This catches subtle attacks where magnitude and direction appear normal but the model impact is anomalous.

Layer 3: Time-Decay Reputation Scoring

SignGuard maintains a reputation score for each client, ranging from 0 (untrusted) to 1 (fully trusted):

```
def update_reputation(old_reputation: float, contribution_score: float,
decay_factor: float = 0.9) -> float:
    return old_reputation * decay_factor + contribution_score * (1 -
decay_factor)
```

Recent behavior is weighted more heavily than historical actions. A client who consistently contributed clean updates but suddenly begins sending poisoned data will see their reputation drop rapidly.

Adaptive thresholds: Clients with higher reputation scores are granted slightly more leniency in anomaly detection. Low-reputation clients face stricter scrutiny.

Reputation recovery: Clients who have been flagged can gradually rebuild trust by sending consistent, verified updates over multiple rounds.

Integration with Aggregation

SignGuard plugs into standard FL aggregation with Byzantine-robust methods:

```
def signguard_aggregate(client_updates):
    valid_updates = []

    for client_id, update, signature in client_updates:
        # Layer 1: Verify signature
        if not verify_signature(client_id, update, signature):
            log_suspicious(client_id, "Invalid signature")
            continue

        # Layer 2: Multi-factor anomaly detection
        if not (magnitude_analysis(update) and
                direction_analysis(update) and
                loss_analysis(update)):
            # Layer 3: Update reputation
            reputation[client_id] *= 0.8  # Penalty
            if reputation[client_id] < threshold:
                log_suspicious(client_id, "Anomaly detected")
                continue

        # Reward good behavior
        reputation[client_id] = min(1.0, reputation[client_id] * 1.05)
        valid_updates.append(update)

    # Byzantine-robust aggregation on verified updates
    return krum_aggregate(valid_updates)
```

Experimental Results

I tested SignGuard against several attacks:

Sign Flipping Attack

```
# Without SignGuard
poisoned = -genuine_update
# Detection: 23% (Krum sometimes catches it)

# With SignGuard (all three layers)
```

```
poisoned = -genuine_update
signature = sign(poisoned)  # Client can sign anything
# Direction analysis catches the flip: 94.5% detection
# Magnitude analysis may miss it (same norm)
# Reputation degrades the malicious client over time
```

Label Flipping Attack

```
# Without SignGuard
# Client sends updates with flipped labels
# Detection: ~15% (hard to detect with simple aggregation)

# With SignGuard
# Loss-based analysis detects unexpected loss increase
# Direction analysis catches divergent gradient direction
# Combined detection: 89.2%
```

Backdoor Attack

```
# Without SignGuard
# Client embeds backdoor in specific gradient dimensions
# Detection: <10% (very subtle)

# With SignGuard
# Direction analysis detects slight divergence
# Reputation system tracks persistent anomalies
# Combined detection: 76.8%
```

Key Insight

Signatures prevent certain attack classes, but anomaly detection and reputation systems catch what signatures miss:

ECDSA Signatures prevent:

- Impersonation attacks
- Replay attacks (with nonces)
- Man-in-the-middle tampering
- Denial of sending (repudiation)

Anomaly Detection + Reputation prevent:

- Data poisoning (through magnitude/loss analysis)
- Model poisoning (through direction analysis)
- Label flipping (through loss analysis)
- Backdoor injection (through direction + reputation tracking)

Honest Limitations

SignGuard provides strong defenses, but it's important to be clear about what it doesn't do:

No privacy guarantees: ECDSA signatures authenticate updates but don't hide their contents. For privacy preservation, you'd need differential privacy or secure aggregation — techniques covered in the broader FL Security Research Suite.

Trusted server assumption: SignGuard assumes the aggregation server is honest. A fully decentralized system would require additional consensus mechanisms.

Adaptive attackers: Sophisticated attackers who slowly poison the model over many rounds may evade detection. The reputation system helps but isn't perfect against patient adversaries.

Non-IID sensitivity: Like all federated systems, SignGuard's effectiveness depends on data distribution across clients. Extremely heterogeneous data can make anomaly detection more challenging.

Performance Metrics

Metric	Value
Attack detection rate	94.5% (combined)
Accuracy degradation (clean data)	<0.2%
Sign flipping detection	94.5%
Label flipping detection	89.2%
Backdoor detection	76.8%
Signature verification time	<5ms per update (P-256)
Reputation convergence	~10 rounds for stable scoring

Implementation Highlights

SignGuard is implemented in Python with the following key components:

Cryptography: `ecdsa` library for P-256 signature generation and verification

Numerical operations: `numpy` for efficient vector operations on model updates

Anomaly detection: Custom implementations of magnitude, direction, and loss-based analysis

Reputation tracking: Time-decay scoring with configurable decay factors

Aggregation: Krum and Multi-Krum algorithms for Byzantine-robust aggregation

The code is structured modularly, allowing each defense layer to be used independently or in combination.

Conclusion

SignGuard demonstrates that cryptographic defenses are practical for federated learning. Signatures add minimal overhead while preventing entire attack classes.

The future of FL security isn't choosing between cryptographic and algorithmic defenses—it's combining them. SignGuard for integrity, robust aggregation for quality.

This layered approach is what I believe will make federated learning production-ready for security-critical applications like banking and healthcare.

See the SignGuard implementation and related Byzantine Robustness projects in my portfolio.

Related Posts

← Back to all posts

Azka Al Azkiyai
AI Security Researcher

© 2026 Azka Al Azkiyai